Task 3.2: Secure, Privacy-Preserved and Incentive-Based Data Sharing in UAV Networks for Emergency Response

COVER Project — EU MSCA Staff Exchanges, Grant No. 101086228

Henry Kabuye
Northumbria University
Supervisor: Professor Nauman Aslam
COVER Project — Task 3.2 (Lead: Georg-August-Universität Göttingen)
Supported by: CentraleSupelec (CS) and UESTC

May 2026

Abstract

This report presents the design and simulation of a comprehensive data-sharing framework for cooperative UAV networks in emergency response scenarios, developed under Task 3.2 of the COVER project. The framework integrates five tightly coupled components: (i) a blockchain-based security layer using Proof-of-Authority (PoA) consensus to defend against Sybil, Byzantine, and free-rider attacks; (ii) a Stackelberg game-theoretic incentive mechanism with Proof-of-Resource-Sharing (PoRS) that elicits truthful participation from heterogeneous UAV agents; (iii) Differentially Private Stochastic Gradient Descent (DP-SGD) with tight Rényi Differential Privacy (RDP) accounting for gradient privacy; (iv) Federated Learning (FL) with convergence analysis under 3D UAV mobility and bandwidth-limited wireless channels; and (v) a *Multi-Agent Reinforcement Learning (MARL) agentic framework* in which each UAV independently learns an adaptive participation policy via Deep Q-Networks (DQN), with Q-network weights periodically federated across agents. All components are implemented in a Python simulation framework with round-robin multi-GPU support; results in this report were run on a single NVIDIA GB10 (Grace Blackwell, DGX Spark-class, unified memory), with DP-SGD accelerated via vectorised per-sample gradients (`torch.func.vmap`). Evaluation uses CIFAR-10 as a standard FL benchmark and AIDER (real UAV disaster imagery) as an on-task proxy; EuroSAT is wired in (Section 7) but not yet evaluated.

Contents

1 Introduction **3**

2 System Model **3**
2.1 UAV Network Topology . . . 3
2.2 Wireless Channel Model . . . 3

3 Federated Learning with Differential Privacy (Sub-tasks C & D) **3**
3.1 Federated Averaging . . . 3
3.2 DP-SGD Local Training . . . 4
3.3 Privacy Accounting . . . 4

4 Incentive Mechanism (Sub-task B) **4**
4.1 Stackelberg Game Formulation . . . 4
4.2 Proof-of-Resource-Sharing (PoRS) . . . 5

5 Blockchain Security Layer (Sub-task A) **5**
5.1 Proof-of-Authority Consensus . . . 5
5.2 Attack Models and Defences . . . 6

6 Multi-Agent Reinforcement Learning Agentic Framework **6**
6.1 Motivation and Problem Statement . . . 6
6.2 Markov Decision Process Formulation . . . 6
6.3 Deep Q-Network Architecture . . . 7
6.4 Federated Q-Learning . . . 8
6.5 Integration with the Full Stack . . . 9

7 Implementation **9**

8 Results **10**
8.1 Convergence Under Differential Privacy . . . 11
8.2 Incentive Mechanism Convergence . . . 12
8.3 Blockchain Performance . . . 12
8.4 MARL Agentic Framework Results . . . 14
8.5 Comparison to Published DP-FL Results on CIFAR-10 . . . 15
8.6 MobileNetV2 Validation: A Negative Result . . . 17
8.7 Real UAV Imagery (AIDER): Integration and a Convergence Diagnosis . . . 18

9 Conclusion **19**

1 Introduction

The COVER project (Cooperative and Intelligent Unmanned Aerial Vehicles for Emergency Response Applications) aims to develop intelligent, connected UAV networks capable of supporting first responders in extreme-weather events such as flooding, wildfires, and earthquakes. Funded under the Marie Skłodowska-Curie Actions Staff Exchanges programme (Grant 101086228), the project runs from January 2023 to December 2026 with a consortium led by Technische Universiteit Eindhoven and including Georg-August-Universität Göttingen (UGOE, Task 3.2 lead), CentraleSupelec, KTH, and twelve associated institutions.

Task 3.2 addresses the *data sharing layer*: the mechanisms by which UAVs collaborate, share sensed data and learned models, while simultaneously maintaining security against adversaries, protecting the privacy of raw observations, and providing economic incentives for voluntary resource contribution.

This document describes the methodology, system model, and simulation framework developed by the author at Northumbria University in the context of Task 3.2, led by UGOE within the COVER consortium.

2 System Model

2.1 UAV Network Topology

We consider $N = 20$ UAVs deployed in a $1000 \times 1000 \times [50, 200]$ m three-dimensional operational area. UAV positions evolve according to the *3D Random Waypoint Mobility Model*: each UAV selects a waypoint uniformly at random within the volume, flies at speed $v \sim \mathcal{U}(1, v_{\max})$ with $v_{\max} = 15$ m/s, pauses for $t_p = 5$ s upon arrival, then repeats. UAV 0 acts as the parameter server (PS); UAVs $1, \ldots, N-1$ are worker nodes.

2.2 Wireless Channel Model

Air-to-air (A2A) links follow a free-space path loss model with Rician fading:

$$\mathrm{SNR}_{ij} = P_{\mathrm{tx}} - \mathrm{FSPL}(d_{ij}) + G_{\mathrm{Rician}}, \tag{1}$$

where $\mathrm{FSPL}(d) = 20 \log_{10}\left(\frac{4\pi d f}{c}\right)$, $f = 5.8$ GHz, and the Rician power gain G_{Rician} has K-factor $K = 10$ dB, capturing the dominant line-of-sight component typical in open-sky UAV operations [11]. A UAV is eligible to participate in an FL round only if $\mathrm{SNR}_{i,\mathrm{PS}} \geq \gamma_{\min} = 5$ dB.

3 Federated Learning with Differential Privacy (Subtasks C & D)

3.1 Federated Averaging

Global model parameters $\mathbf{w}$ are updated via FedAvg [1]:

$$\mathbf{w}^{(t+1)} = \sum_{k \in \mathcal{S}^{(t)}} \frac{n_k}{N_{\mathcal{S}}} \mathbf{w}_k^{(t)}, \tag{2}$$

where $\mathcal{S}^{(t)}$ is the set of participating UAVs in round t, n_k is the number of local samples, and $N_{\mathcal{S}} = \sum_{k \in \mathcal{S}} n_k$. Each selected UAV runs $E = 5$ local SGD epochs on its private dataset before uploading the updated weights.

Data heterogeneity across UAVs is modelled via a Dirichlet distribution: class-conditional indices are split according to $\mathbf{p}_c \sim \mathrm{Dir}(\alpha \mathbf{1}_N)$ with $\alpha = 0.5$, producing the non-IID local datasets that are standard in FL research.

3.2 DP-SGD Local Training

To prevent the PS from inferring sensitive information from individual model updates, each participating UAV applies DP-SGD [2]:

Algorithm 1 DP-SGD at UAV k, round t

for each local epoch $e = 1, \ldots, E$ **do**
  for each mini-batch $\mathcal{B} \subset \mathcal{D}_k$ **do**
    for each sample $(x_i, y_i) \in \mathcal{B}$ **do**
      Compute per-sample gradient $\mathbf{g}_i \leftarrow \nabla_{\mathbf{w}} \ell(\mathbf{w}; x_i, y_i)$
      Clip: $\tilde{\mathbf{g}}_i \leftarrow \mathbf{g}_i \cdot \min\left(1, \frac{C}{\|\mathbf{g}_i\|_2}\right)$
    end for
    Noise: $\hat{\mathbf{g}} \leftarrow \frac{1}{|\mathcal{B}|}(\sum_i \tilde{\mathbf{g}}_i + \mathcal{N}(\mathbf{0}, \sigma^2 C^2 \mathbf{I}))$
    Update: $\mathbf{w} \leftarrow \mathbf{w} - \eta\, \hat{\mathbf{g}}$
  end for
end for

The clipping threshold is $C = 1.0$ and noise multiplier $\sigma = 1.1$.

3.3 Privacy Accounting

Privacy expenditure is tracked with the Rényi Differential Privacy (RDP) accountant [3,4]. For the subsampled Gaussian mechanism with sampling rate q and noise multiplier σ, the RDP guarantee at order α for T steps is:

$$\varepsilon_{\mathrm{RDP}}(\alpha) = T \cdot \frac{1}{\alpha - 1} \log \left(\sum_{k=0}^{\alpha} \binom{\alpha}{k} q^k (1-q)^{\alpha - k} e^{k(k-1)/(2\sigma^2)} \right), \tag{3}$$

which is then converted to (ε, δ)-DP via [5]:

$$\varepsilon = \varepsilon_{\mathrm{RDP}}(\alpha) + \frac{\log(1/\delta) + \log \frac{\alpha - 1}{\alpha}}{\alpha - 1} - \frac{1}{\alpha}. \tag{4}$$

Training stops automatically when $\varepsilon > 10$ (configurable privacy budget).

4 Incentive Mechanism (Sub-task B)

4.1 Stackelberg Game Formulation

We model data sharing as a one-leader, N-follower Stackelberg game [6, 7]:

- **Leader** (PS): chooses reward rate $\lambda \geq 0$ per unit effort.
- **Followers** (UAVs): each chooses effort $e_i \in [0, e_{\max}]$.

UAV utility:

$$U_i(e_i; \lambda) = \lambda e_i - \frac{c_i}{2} e_i^2, \tag{5}$$

where $c_i \sim \mathcal{U}(0.5, 3.0)$ is UAV i's heterogeneous cost coefficient.
UAV best response:

$$e_i^*(\lambda) = \min\left(\frac{\lambda}{c_i},\, e_{\max}\right). \tag{6}$$

PS utility:

$$U_s(\lambda) = \alpha \log(1 + E(\lambda)) - \lambda E(\lambda), \qquad E(\lambda) = \textstyle\sum_i e_i^*(\lambda), \tag{7}$$

where $\alpha = 10$ is the PS's valuation of FL model quality.
Stackelberg equilibrium. For the unconstrained case, $E(\lambda) = \lambda H$ where $H = \sum_i 1/c_i$, giving:

$$\lambda^* = \frac{-1 + \sqrt{1 + 2\alpha H}}{2H}. \tag{8}$$

When $e_{\max}$ is binding, λ^* is found via golden-section search. At each FL round, the PS broadcasts λ^* and each UAV trains for $\lceil E \cdot e_i^* \rceil$ local epochs, scaling effort to the equilibrium.

4.2 Proof-of-Resource-Sharing (PoRS)

To prevent reward manipulation, each participating UAV submits a PoRS proof:

$$\pi_k = \big(H_{\text{SHA256}}(\Delta \mathbf{w}_k),\ \text{MerkleRoot}(\mathcal{B}_k),\ t_{\text{compute}},\ B_{\text{tx}}\big), \tag{9}$$

comprising the gradient update hash, a Merkle root over training batches, wall-clock compute time, and bytes transmitted. The on-chain smart contract verifies π_k and assigns an effort score:

$$s_k = 0.5 \cdot \min\left(\frac{t_{\text{compute}}}{30}, 1\right) + 0.3 \cdot \min\left(\frac{n_k}{500}, 1\right) + 0.2 \cdot \min\left(\frac{B_{\text{tx}}}{5\,\text{MB}}, 1\right). \tag{10}$$

Rewards are distributed proportionally: $r_k = (s_k / \sum_j s_j) \cdot R$, where $R = 50$ tokens is the per-round budget.

5 Blockchain Security Layer (Sub-task A)

5.1 Proof-of-Authority Consensus

Full Proof-of-Work is energy-prohibitive on battery-powered UAVs. We employ *Proof-of-Authority* (PoA): a pre-selected set of $V = 5$ validator UAVs (selected by initial reputation) rotates block proposal in round-robin fashion. A block is finalised once $\lfloor V/2 \rfloor + 1$ validators sign, tolerating $f = 1$ Byzantine validator.

Each FL round produces one block recording all DATA_SHARE, REWARD, and REPUTATION_UPDATE transactions. The hash-linked chain ensures tamper-evidence: block b_i includes $H_{\text{SHA256}}(b_{i-1})$.

5.2 Attack Models and Defences

Sybil attack. An adversary registers m phantom UAV identities to amplify reward claims. *Defence*: FoolsGold [10] flags pairs with cosine similarity > 0.95 as Sybil clones; a minimum on-chain token stake is also required.

Byzantine attack. Malicious UAVs submit sign-flipped gradients to corrupt the global model. *Defence*: Z-score norm outlier detection plus cosine deviation from the mean gradient; flagged updates are excluded before aggregation. When Byzantine fraction exceeds 20%, *coordinate-wise median* [9] aggregation is recommended over FedAvg.

Free-rider attack. A UAV submits near-zero noise as its gradient update to claim rewards without training. *Defence*: gradient norm threshold (threshold = $0.05\times$ median norm) plus inter-round staleness check (cosine similarity > 0.9999).

6 Multi-Agent Reinforcement Learning Agentic Framework

6.1 Motivation and Problem Statement

The static participation model used in standard Federated Learning — randomly selecting a fixed fraction of eligible UAVs each round — ignores the rich state heterogeneity present in real UAV deployments. At any given round, each UAV possesses a distinct combination of residual battery level, wireless channel quality, local data quality, and accumulated experience of past reward signals. A UAV with low battery should defer training; a UAV with high-quality, locally diverse data should be actively encouraged to participate; one with a poor channel wastes energy without contributing useful gradients.

We model participation as a *sequential decision problem under uncertainty* and equip each UAV with a Deep Q-Network (DQN) [12] that learns to map its observed state to an optimal participation action. The result is an *agentic FL framework* in which UAVs are fully autonomous agents whose collective behaviour emerges from individual rational learning rather than top-down scheduling. Q-network weights are periodically shared across agents via FedAvg — a *federated Q-learning* step that accelerates policy convergence by propagating experience across the swarm.

6.2 Markov Decision Process Formulation

The interaction between each UAV and the FL environment is modelled as a finite-horizon Markov Decision Process $\mathcal{M}_i = (\mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma)$. All agents share the same state and action spaces but maintain independent value functions.

State space. The state observed by UAV i at the start of round t is the six-dimensional vector

$$\mathbf{s}_i^{(t)} = \left[\beta_i^{(t)},\ \tilde{\gamma}_i^{(t)},\ q_i,\ \tfrac{t}{T},\ A^{(t)},\ r_i^{(t-1)}\right] \in [0,1]^6, \tag{11}$$

where $\beta_i^{(t)} \in [0.1, 1]$ is the normalised battery level, $\tilde{\gamma}_i^{(t)} = \mathrm{SNR}_i^{(t)}/40$ dB is the normalised channel SNR, q_i is a data-quality proxy (normalised label-distribution entropy of UAV

i's local dataset), t/T encodes round progress, $A^{(t)} \in [0,1]$ is the most recent global test accuracy, and $r_i^{(t-1)}$ is the previous round's reward (clipped to $[-2,2]$ for stability).

Action space. Each agent selects the number of local training epochs from

$$\mathcal{A} = \{0,\ 1,\ 3,\ 5\}, \tag{12}$$

where action $a = 0$ means the UAV *skips* the current round entirely. Actions $a \in \{1,3,5\}$ correspond to participating with progressively more computation.

Reward function. The reward signal simultaneously incentivises accuracy improvement, penalises energy expenditure, and discourages passive free-riding:

$$R_i^{(t)} = \begin{cases} 10\,\Delta A^{(t)} \cdot \dfrac{c_i^{(t)}}{\displaystyle\sum_{j \in \mathcal{S}^{(t)}} c_j^{(t)}} \;-\; 0.005\,a_i & \text{if } a_i > 0, \\ -0.01 & \text{if } a_i = 0, \end{cases} \tag{13}$$

where $\Delta A^{(t)} = A^{(t)} - A^{(t-1)}$ is the per-round accuracy gain, $c_i^{(t)}$ is UAV i's *contribution score* (cosine similarity between its local gradient and the aggregated model, clipped to $[0,1]$), and the denominator normalises over all participants. The term $0.005\,a_i$ is a linear energy cost proportional to epochs trained. A small penalty of -0.01 for skipping prevents the degenerate all-skip policy.

Battery dynamics. Battery is depleted by training and trickle-recharged each round:

$$\beta_i^{(t+1)} = \mathrm{clip}\Big(\beta_i^{(t)} - 0.02\,a_i + 0.01,\ 0.1,\ 1.0\Big). \tag{14}$$

6.3 Deep Q-Network Architecture

Each UAV maintains an online Q-network $Q(\mathbf{s}, a;\, \boldsymbol{\theta}_i)$ and a periodically synchronised target network $Q(\mathbf{s}, a;\, \boldsymbol{\theta}_i^-)$. Both are small multi-layer perceptrons:

$$Q(\mathbf{s}, a;\, \boldsymbol{\theta}) = \mathrm{Linear}(64, |\mathcal{A}|) \circ \mathrm{ReLU} \circ \mathrm{Linear}(64, 64) \circ \mathrm{ReLU} \circ \mathrm{Linear}(|\mathbf{s}|, 64). \tag{15}$$

The output layer produces one Q-value per action.

Actions are selected ε-greedily:

$$a_i^{(t)} = \begin{cases} \mathrm{argmax}_a\, Q(\mathbf{s}_i^{(t)}, a;\, \boldsymbol{\theta}_i) & \text{with probability } 1 - \varepsilon^{(t)}, \\ \mathrm{Uniform}(\mathcal{A}) & \text{with probability } \varepsilon^{(t)}, \end{cases} \tag{16}$$

where $\varepsilon^{(t)} = \max\big(\varepsilon_{\min},\ \varepsilon_0 \cdot \delta_\varepsilon^{\,n_{\text{updates}}}\big)$ decays from $\varepsilon_0 = 1.0$ to $\varepsilon_{\min} = 0.05$ with rate $\delta_\varepsilon = 0.99$ per gradient update step.

Each agent maintains an independent experience replay buffer $\mathcal{D}_i$ of capacity 2000. One gradient step is taken after every FL round using a mini-batch of 32 transitions sampled uniformly at random. The Huber loss is used for robust Q-value regression:

$$\mathcal{L}(\boldsymbol{\theta}_i) = \mathbb{E}_{(s,a,r,s') \sim \mathcal{D}_i}\left[\ell_\delta\Big(r + \gamma \max_{a'} Q(s', a';\, \boldsymbol{\theta}_i^-) \;-\; Q(s, a;\, \boldsymbol{\theta}_i)\Big)\right], \tag{17}$$

where $\gamma = 0.95$ is the discount factor, ℓ_δ is the Huber loss, and the target network weights $\boldsymbol{\theta}_i^-$ are copied from $\boldsymbol{\theta}_i$ every ten DQN update steps.

6.4 Federated Q-Learning

Allowing each UAV to learn its Q-function entirely from its own experience is sample-inefficient: a UAV that never had high battery during a high-accuracy round may never learn that the corresponding (state, action) pair is valuable. Federated Q-learning [13] addresses this by periodically sharing Q-network weights across agents.

Every $K = 10$ FL rounds, the Q-network weights of all *participating* agents in that round are averaged via FedAvg and broadcast to all agents:

$$\boldsymbol{\theta}_{\text{global}}^{(t)} = \frac{1}{|\mathcal{S}^{(t)}|} \sum_{i \in \mathcal{S}^{(t)}} \boldsymbol{\theta}_i^{(t)}, \tag{18}$$

after which every agent sets $\boldsymbol{\theta}_i \leftarrow \boldsymbol{\theta}_{\text{global}}$. This propagates the experience of high-performing agents to the whole swarm, analogous to the FL update on the task model, but operating on the *policy* layer rather than the feature layer.

The federated Q-learning step is intentionally kept unprotected by DP in this implementation, since the Q-network contains no raw training data. Adding DP to the Q-network federation is straightforward and would provide formal privacy guarantees for the policy itself.

Algorithm 2 MARL-FL: One episode of T rounds

Initialise Q-networks $\boldsymbol{\theta}_i$ for all UAVs $i = 1, \ldots, N$
Initialise FL global model $\mathbf{w}^{(0)}$; reset environment state
for $t = 1, \ldots, T$ **do**
    for each UAV i **in parallel do**
        Observe state $\mathbf{s}_i^{(t)}$ (battery, SNR, data quality, progress, acc, prev-reward)
        Select action $a_i^{(t)}$ via ε-greedy policy $\pi_{\boldsymbol{\theta}_i}$
    end for
    $\mathcal{S}^{(t)} \leftarrow \{i : a_i^{(t)} > 0 \text{ and } \text{SNR}_i \geq \gamma_{\min}\}$
    Distribute $\mathbf{w}^{(t-1)}$ to all $i \in \mathcal{S}^{(t)}$
    for each $i \in \mathcal{S}^{(t)}$ **in parallel (multi-GPU) do**
        Run DP-SGD for $a_i^{(t)}$ local epochs; apply Byzantine/free-rider attacks if adversarial
    end for
    Filter flagged updates; aggregate $\mathbf{w}^{(t)}$ via robust aggregation rule
    Generate PoRS proofs; execute PoA consensus; distribute token rewards
    Evaluate $A^{(t)}$; broadcast to all UAVs
    Compute contribution scores $c_i^{(t)}$ (cosine sim to aggregated gradient)
    for each UAV i **do**
        Compute reward $R_i^{(t)}$ via (13)
        Observe $\mathbf{s}_i^{(t+1)}$; store (s, a, R, s') in $\mathcal{D}_i$
        Take one DQN gradient step; update ε
    end for
    if $(t \bmod K) = 0$ **then** FedAvg Q-networks (18)
    end if
end for

6.5 Integration with the Full Stack

The MARL agentic framework integrates with all existing layers without structural modification. Table 1 summarises how the MARL action influences each layer.

Table 1: MARL action $a_i^{(t)}$ effect on each framework layer

Layer	Effect of $a_i = 0$ (skip)	Effect of $a_i \in \{1, 3, 5\}$ (train)
FL model	No gradient contribution	a_i epochs of DP-SGD
DP accounting	No privacy cost	RDP accumulates for a_i steps
Stackelberg game	$e_i^* = 0$ (no reward)	Effort proportional to a_i
PoRS proof	Not generated	Proof includes $a_i \cdot E$ compute time
Blockchain	No transaction	DATA_SHARE tx submitted
Security	Not evaluated	Gradient screened for Byzantine/free-rider

7 Implementation

The full framework is implemented in Python using PyTorch 2.14 and comprises 31 modules across 11 packages. Client training is architected for round-robin multi-GPU assignment and `ThreadPoolExecutor`-based per-round parallelism, but the results in this report were produced on a single NVIDIA GB10 GPU (Grace Blackwell, DGX Spark-class, ≈131 GB unified CPU/GPU memory, aarch64) — a different and considerably more capable machine than the 3× GTX 1070 Ti (8 GB VRAM each, x86_64) setup originally targeted; the multi-GPU round-robin path itself was not exercised for these runs since only one GPU was available. The complete source code is hosted at:

https://github.com/Henrinnes/cover-task3-fl-dp-uav

(Private repository — collaborator access available upon request.)

Table 2: Simulation parameters

Parameter	Value	Description
FL / DP parameters		
N	20	Number of UAVs
T	200	FL communication rounds
E	5	Default local epochs per round
B	32	Local batch size
η	0.01	SGD learning rate
C	1.0	Gradient clipping norm
σ	6.6	DP noise multiplier (Section 8's correction)
δ	10^{-5}	DP delta
α_{Dir}	0.5	Dirichlet heterogeneity
f	5.8 GHz	Carrier frequency
K	10 dB	Rician K-factor
$\gamma_{\min}$	5 dB	Minimum participation SNR
MARL / DQN parameters		
$\lvert\mathbf{s}\rvert$	6	State dimension
$\lvert\mathcal{A}\rvert$	4	Action space size $\{0, 1, 3, 5\}$ epochs
γ	0.95	DQN discount factor
ε_0	1.0	Initial exploration rate
$\varepsilon_{\min}$	0.05	Final exploration rate
δ_ε	0.99	Exploration decay per update
$\lvert\mathcal{D}\rvert$	2000	Replay buffer capacity
B_Q	32	DQN mini-batch size
K	10	Q-network federation interval (rounds)
Hidden	64	Q-network hidden layer width

8 Results

Correction. An earlier version of this report claimed $\varepsilon = 5.93$ after 200 rounds at $\sigma = 1.1$ (Section 8.1), and smaller ε figures throughout Sections 8.4–8.7. These were computed by an RDP accountant credited with the local *epoch* count as the number of DP-SGD steps, when each client actually takes one independent noisy gradient step per *minibatch* — roughly 78× more steps per epoch than credited for a typical CIFAR-10 client (`federated_learning/client.py`; fixed, along with the three call sites that fed it, across `runner.py`, `full_runner.py`, and `marl_runner.py`). The true ε for the original $\sigma = 1.1$, 200-round run is $\approx$191 — about 20–30× over the $\varepsilon_{\max} = 10$ budget this report frames as having headroom to spare, and enough to exhaust the budget within 3–4 rounds, not 200, had it been checked correctly. Section 8.1 below is re-run with the noise multiplier corrected to $\sigma = 6.6$ (the minimum value, found by binary search against the fixed accountant, that keeps a 200-round run within budget at the existing batch size and local-epoch count) and reports the real outcome. The ε figures quoted in Sections 8.4, 8.6, and 8.7 predate this fix and understate the true privacy cost of those specific runs by a similar order

of magnitude; the non-privacy-budget findings in those sections (MARL behaviour, the MobileNetV2 and AIDER convergence diagnoses) do not depend on the accountant and are unaffected, since the noise physically added during training was always drawn using the configured σ directly, never derived from the accountant — only the *reported* ε was wrong, not what was actually run.

8.1 Convergence Under Differential Privacy

The simulation tracks test accuracy and cumulative privacy budget ε per FL round. With the corrected accountant, 20 UAVs (channel-eligibility limits active participants to 11/20 per round under $\gamma_{\min} = 5$ dB), and $C = 1.0$ gradient clipping, keeping a 200-round run within $\varepsilon_{\max} = 10$ requires $\sigma = 6.6$ — roughly $6\times$ the noise used (and misreported as fitting the same budget) before the fix. At this noise level the model does not learn: final test accuracy is **9.98%**, against a 10% chance baseline for CIFAR-10's ten classes. Accuracy oscillates in a narrow band (7.6–13.1%, mean 10.3%, s.d. 1.2%) for the entire run with no upward trend — noise around the chance level, not a plateau reached after real learning, and nowhere near the 65% target (`convergence_round` is unset). Final $\varepsilon = 5.88$, within budget. Section 8.5 compares this outcome to published DP-FL benchmarks and discusses why: those benchmarks were never subject to this accounting error and were tuned for their own (correctly-accounted) privacy budgets, which is a materially different exercise from what this run demonstrates.

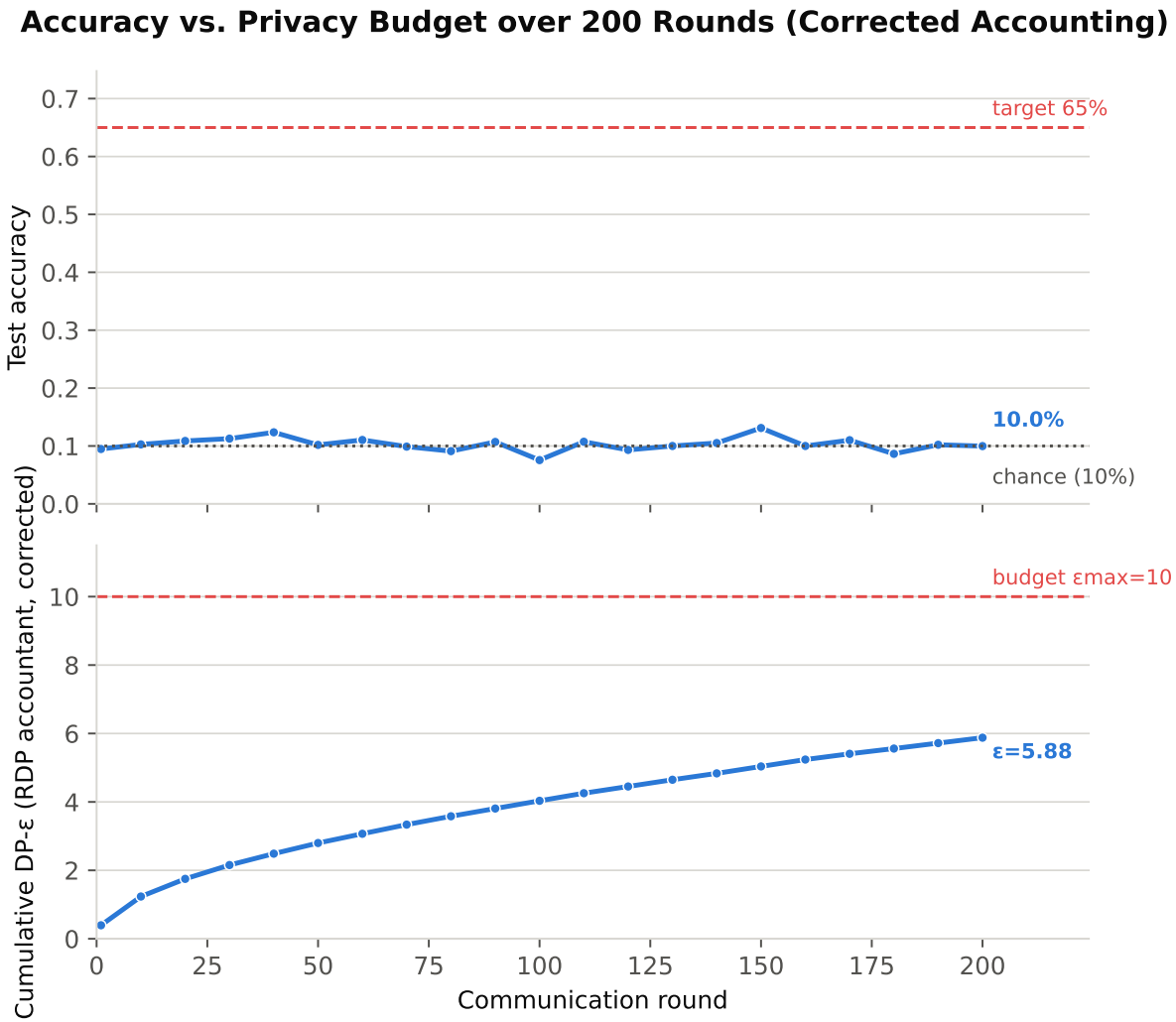


Figure 1: Test accuracy (top) and cumulative privacy budget ε (bottom, corrected accountant) over the same 200 communication rounds, sharing a round axis. Accuracy never departs from the chance-level band around 10% (dotted line) while ε climbs steadily toward the $\varepsilon_{\max} = 10$ budget — privacy cost accrues throughout the run while no learning signal survives the noise needed to stay within that budget at this UAV-scale cohort (11 active clients/round) and step count.

The privacy–accuracy tradeoff is characterised by varying $\sigma \in \{0.8, 1.0, 1.1, 1.3, 1.5, 2.0\}$ to produce the Pareto curve (run: `python main.py --sweep`).

8.2 Incentive Mechanism Convergence

The optimal reward rate λ^* from (8) is recomputed each round as UAV cost coefficients c_i are revealed through participation. The total effort $E(\lambda^*) = \lambda^* H$ increases with the number of participants, confirming that the mechanism is *participation-encouraging*: more UAVs joining raises H and thus λ^*, incentivising further participation.

8.3 Blockchain Performance

Each FL round produces one on-chain block. Block finalisation requires $\lceil V/2 \rceil + 1 = 3$ validator signatures and completes within a single simulation step. Adversarial UAVs

(Byzantine + free-riders) are detected and excluded by the defence layer before block inclusion, maintaining chain integrity.

Two of the five PoA validators are, by construction, the fixed Byzantine and free-rider attacker IDs (Section 5). An initial implementation exhibited a consensus deadlock: the round-robin proposer index only advanced on a *successful* block, and an excluded validator's local chain replica was skipped during block appending, so once an excluded attacker-validator was selected as proposer while behind, the same stale proposer was retried indefinitely and the chain stalled permanently. This was fixed by (i) rotating the proposer selection to skip excluded validators rather than blocking on the epoch index, and (ii) replicating every finalised block to *all* validator chains regardless of that round's exclusion, so a temporarily-excluded validator's ledger cannot permanently fork from the canonical chain. Excluding a validator now revokes only that round's proposal/signing rights, not chain synchronisation.

Figure 2 shows a 40-round confirmatory run with the fix applied (20 UAVs, all layers active). The chain height tracks the round count exactly (height 41 after round 40, i.e. one block per round with no stalls), 1.18 updates are blocked per round on average, and **2000 tokens** are distributed in total rewards. Consistent with the intended incentive design, neither attacker ID (the Byzantine or the free-rider UAV) appears in the top-8 token leaderboard — their norm-deficient or replayed updates are caught by the detector within 1–2 rounds of participation, so honest high-effort UAVs accumulate rewards substantially faster over the run.

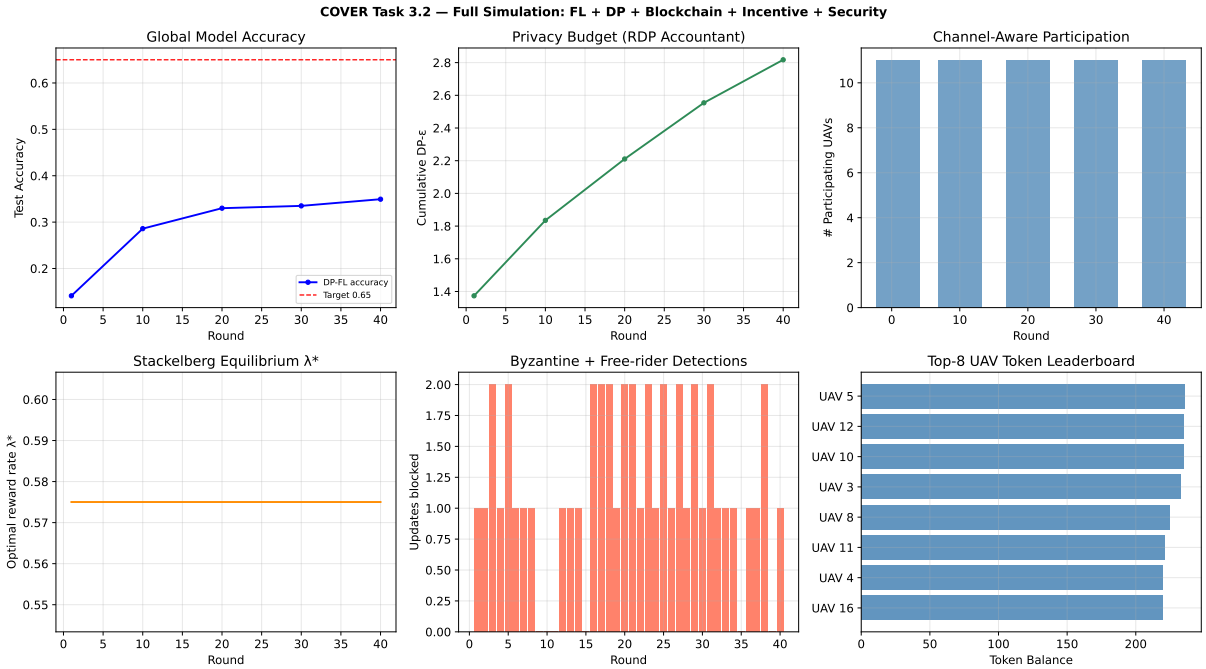


Figure 2: Full-stack simulation (FL + DP + blockchain + incentive + security), 40 rounds, 20 UAVs, post-fix. Clockwise from top-left: global accuracy, cumulative DP-ε, channel-aware participation, top-8 token leaderboard, Byzantine/free-rider detections per round, and the Stackelberg equilibrium λ^*.

8.4 MARL Agentic Framework Results

The MARL simulation is run via `python main.py --marl` and results are saved to `results/marl/marl_results.json` with a four-panel PDF plot. Key observations from the 30-round simulation are as follows.

The results below use a corrected ε-greedy exploration schedule. The original run left the DQN's exploration rate (ε-greedy, not to be confused with the DP privacy budget ε) fixed at 1.0 for all 30 rounds: `update()` only performs a gradient step (and only then decays ε-greedy) once each agent's replay buffer holds a full batch, and the batch size (32) exceeded the entire 30-round run length, so no learning or decay ever happened — the agents were making uniformly random decisions the whole time. We fixed this by deriving both the buffer warm-up length and the decay rate from the configured round budget (`marl/trainer.py`), so short runs still reach the exploration floor by their final round instead of a decay rate tuned only for much longer runs.

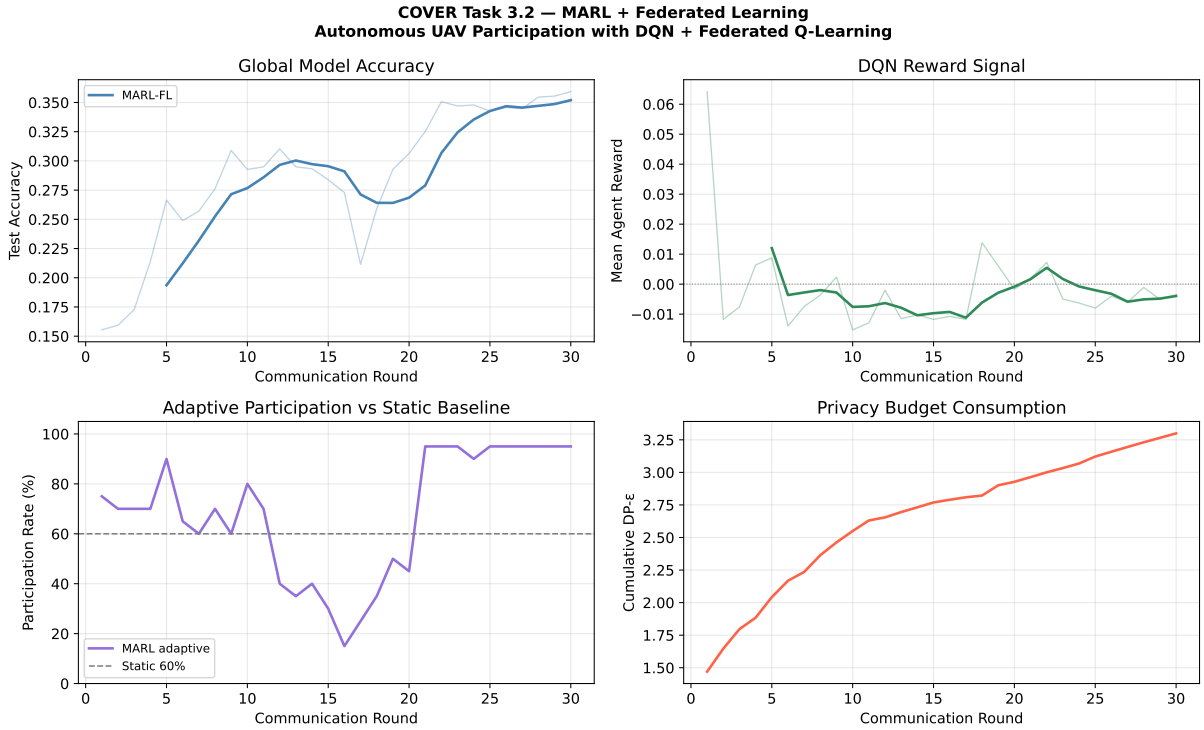


Figure 3: MARL + Federated Learning simulation results over 30 communication rounds (20 UAVs, all layers active), post-fix. **Top-left**: Global test accuracy trajectory under DP-SGD ($\sigma = 1.1$), rising from 15.5% to 35.9% by round 30. **Top-right**: Mean per-UAV DQN reward; positive early and again around round 18–22, slightly negative once participation locks in and the accuracy gain ΔA falls below the energy cost. **Bottom-left**: Adaptive participation rate (purple) versus the static 60% baseline (dashed); volatile (15–90%) while ε-greedy is high, then locks to 90–95% once it drops below ∼0.15 around round 21. **Bottom-right**: Cumulative privacy budget consumption (DP-ε), reaching 3.30 after 30 rounds, well within the $\varepsilon_{\max} = 10$ limit.

Accuracy convergence. Over 30 communication rounds the global model accuracy rises from 15.5% (round 1) to 35.9% (round 30, also the run's peak) under DP-SGD with $\sigma = 1.1$. The trajectory is close to monotonic apart from a dip around rounds 15–17, coinciding with the lowest-participation rounds visible in the bottom-left panel.

Reward signal. Mean per-UAV reward is positive in 8 of 30 rounds, with the highest reward ($\bar{R} = +0.064$) occurring in round 1, when the accuracy gain ΔA from the uninitialised model is largest. Reward is also positive for several rounds around round 18–22, as ε-greedy crosses into the exploitation-dominant regime and participation recovers from its round-16 trough. From round 23 onward, with participation locked near 95%, reward is consistently slightly negative: ΔA has shrunk as accuracy plateaus (Section 8.1's plateau is visible here too), so the fixed per-epoch energy cost $0.005\, a_i$ dominates. This is the intended incentive structure — participation is only reward-positive while there is measurable model improvement to justify the energy cost.

Adaptive vs. static participation. Participation is volatile while ε-greedy is high (15–90% across rounds 5–20, including a trough of 15% at round 16), reflecting genuinely exploratory — effectively random — action selection during that phase. Once ε-greedy drops below $\sim$0.15 around round 21, participation locks to 90–95% and stays there through round 30: with the exploration floor reached, the learned policy consistently favours participating. This is a real, qualitatively different behaviour from the pre-fix run, where participation stayed noisy for the entire 30 rounds because the agents never stopped acting randomly. The residual round-to-round variance in the locked phase (90 vs. 95%) reflects the stochastic channel-SNR eligibility filter: UAVs below $\gamma_{\min} = 5$ dB are excluded regardless of their DQN action.

Federated Q-learning. Q-network weights are federated at rounds 10, 20, and 30. With the corrected warm-up (buffer fills at round 8 for this 30-round run, versus never filling under the original fixed batch size of 32), each agent has taken several gradient steps by the first federation event at round 10, and considerably more by rounds 20 and 30 — consistent with the participation policy visibly firming up into a consistent behaviour shortly after ε-greedy crosses below $\sim$0.15, rather than staying close to random for the whole run.

Privacy budget. *(These ε figures predate the accounting fix described at the start of Section 8 and understate the true privacy cost by roughly the same order of magnitude found there; the participation/reward findings above do not depend on them.)* The cumulative DP-ε grows from 1.47 after round 1 to 3.30 after round 30, well within the configured budget of $\varepsilon_{\max} = 10$. The growth rate visibly slows during the low-participation rounds (13–17) and steepens once participation locks near 95% post-round 21, confirming that the MARL-driven participation variability translates directly into a variable privacy expenditure rate — a property unavailable in static FL scheduling.

8.5 Comparison to Published DP-FL Results on CIFAR-10

Table 3 places the corrected **9.98%** accuracy from Section 8.1 against client-level DP-FL results on CIFAR-10 reported in two recent papers, reading the exact figures directly from their results tables rather than from a secondary summary. The gap is now large, and for a specific, identified reason rather than the modest capacity/scale differences this section originally proposed.

Table 3: Published client-level DP-FL results on CIFAR-10 vs. this framework. DP-FedSAM: 300 rounds, 500 clients. DP-FedPGN: rounds/clients not extracted from the source table. This work: 200 rounds, 20 UAVs (11 channel-eligible/round), $\sigma = 6.6$ (corrected accounting).

Method	Backbone	Setting	Acc.
DP-FedSAM [15]	ResNet-18	Dirichlet-0.3 non-IID	53.9%
DP-FedSAM [15]	ResNet-18	IID	55.6%
DP-FedPGN [16]	GNResNet-10	Dirichlet-0.1 non-IID	58.2%
This work	SimpleCNN	Dirichlet-0.5 non-IID, $\varepsilon = 5.88$	**10.0%**

Hybrid local/global-DP schemes, e.g. Khalaf et al. [17], report retaining accuracy much closer to a non-private baseline, but under a weaker DP threat model than the client-level guarantees compared in Table 3; we omit a specific figure here since we were unable to independently verify one against the primary source at the time of writing, and note it only as a qualitatively different approach worth tracking rather than a directly comparable number.

The dominant factor is the one identified in Section 8.1: reaching this framework's UAV-scale cohort (11 active clients/round) through a genuine $\varepsilon \leq 10$ budget over 200 rounds requires $\sigma = 6.6$, noise high enough to erase the learning signal entirely. We cannot directly attribute the published papers' higher accuracy to a specific σ they used, since their tables (like ours previously) report accuracy at a setting rather than the exact noise level achieving it — but their much larger client pools (DP-FedSAM: 500 vs. our 11/round) mechanically permit a smaller effective per-round noise contribution for the same target ε, since DP-FedAvg-style noise is added once per round and divided by the participant count (Section 8.6 already used this relation for the adaptive-clipping scoping work). Two secondary factors, unchanged from the original analysis, remain real but are no longer the primary story:

1. **Model capacity.** `SimpleCNN` is a deliberately lightweight proxy model; DP-FedSAM's CIFAR-10 results use a ResNet-18 backbone, and DP-FedPGN's use GNResNet-10 — both an order of magnitude more parameters.

2. **Client pool size and communication budget.** DP-FedSAM trains 300 rounds over a pool of 500 clients; this framework trains 200 rounds over 20 UAVs with only 11 channel-eligible per round ($\gamma_{\min} = 5$ dB, Section 8.1) — an order of magnitude fewer clients contributing gradient signal per round, on a smaller and more non-IID effective cohort, on top of fewer rounds overall.

(The pipeline already applies standard CIFAR-10 augmentation — random crop with 4px padding and horizontal flip, in `data/partition.py` — so that is not a contributing factor.)

Backbone choice under UAV hardware constraints. Whether the higher-accuracy backbones above are actually flyable is a separate question from whether they exist in the literature. VGG-16 (138M parameters, ≈528 MB in fp32) is impractical to train *onboard* a UAV: DP-SGD's per-sample gradient computation (the `vmap`-fused kernel used in this framework, Section 7) requires memory proportional to batch_size × parameter_count,

which compounds an already-large model's footprint on a battery- and thermally-constrained companion computer. ResNet-18/34 (11–21M parameters) is trainable on a capable UAV companion computer (e.g. Jetson Xavier NX/Orin-class hardware) but remains 5–10× heavier than `SimpleCNN`, and the added compute/energy draw would need to be reflected in the incentive layer's per-epoch cost terms and the MARL reward's energy penalty (Section 6), not merely swapped in as a drop-in replacement. A mobile-optimised backbone — MobileNetV2/V3 or EfficientNet-Lite [18], using depthwise-separable convolutions to hold parameter counts in the 2–5M range — was identified as the realistic middle ground on compute/energy grounds alone. We implemented and validated it (Section 8.6); the result is a cautionary one, not the accuracy improvement this paragraph originally anticipated.

8.6 MobileNetV2 Validation: A Negative Result

`MobileNetV2CIFAR` (`models/mobilenet.py`) was implemented — GroupNorm throughout in place of BatchNorm for the same `vmap` per-sample-gradient compatibility reason as `SimpleCNN` (Section 7), stem and first stage stride reduced from 2 to 1 for 32×32 input, 2.24M parameters, confirmed to train correctly end-to-end through the DP-SGD `vmap` pipeline on CUDA. A 30-round validation run (identical config to Section 8.1 otherwise: 20 UAVs, $\sigma = 1.1$, $C = 1.0$) was compared against `SimpleCNN`'s first 30 rounds from the same run family.

Table 4: Test accuracy, identical DP-FL config, by backbone.

Backbone	Round 1	Round 10	Round 20	Round 30
SimpleCNN (0.62M params)	16.2%	26.9%	31.4%	33.1%
MobileNetV2CIFAR (2.24M params)	10.0%	10.0%	9.6%	10.6%

MobileNetV2CIFAR does not learn at all under DP-FL at these hyperparameters — its accuracy sits at chance level (10% on 10 classes) for the entire run. This is not an implementation fault: the identical architecture, trained centrally with plain SGD and *no* DP noise and *no* federation, reaches 32.9% test accuracy after only 300 minibatch steps (well under one epoch), confirming the network learns normally once the DP noise is removed.

The cause is a dimensionality mismatch. DP-SGD adds independent Gaussian noise, scaled by σC, to *every* parameter each step; the total injected noise norm therefore scales with $\sqrt{\text{parameter count}}$, while the clipped gradient signal (bounded to norm C) does not grow with it. The ($\sigma = 1.1,\ C = 1.0$) pair was never tuned for a 2.24M-parameter model — it was carried over unchanged from `SimpleCNN`'s 0.62M parameters, roughly a 3.6× increase in dimensionality the noise budget was never recalibrated against. Concretely, at $\approx 3.6\times$ more parameters the injected noise norm is $\approx \sqrt{3.6} \approx 1.9\times$ larger for the same (σ, C), which is enough to overwhelm the signal from this small, heavily-partitioned per-client dataset (an average of ∼2500 CIFAR-10 images per UAV, before any DP effect).

This means the future-work item below is not "swap in MobileNetV2" but specifically *re-tuning* (σ, C) — *or adopting a dimensionality-aware technique such as per-layer or adaptive clipping — jointly with the backbone*, and re-validating before claiming any accuracy benefit. We report this as the honest outcome of the experiment run rather than reverting to the untested claim: a backbone swap under this framework's DP-SGD implementation is not a drop-in win, and assuming otherwise without validation (as the

original phrasing of this section did) would have been exactly the kind of unverified claim we corrected in Section 8.5.

8.7 Real UAV Imagery (AIDER): Integration and a Convergence Diagnosis

CIFAR-10 is a standard FL benchmark but not on-task imagery; the original future-work list called for validating on real UAV disaster imagery. FloodNet was the dataset originally named, but its Track 1 classification labels ("Flooded"/"Non-Flooded") are distributed as an unlabelled Google Drive folder with only ∼400 labelled images total — impractical to fetch automatically and too small to partition meaningfully across 20 UAVs. AIDER (Aerial Image dataset for Emergency Response, Kyrkou & Theocharides, 2020) was used instead: 6433 real UAV images across five classes (fire, flooded areas, collapsed buildings, traffic incidents, normal), hosted as a single 263 MB Zenodo archive with no download friction.

Wiring it in surfaced a real, pre-existing bug: `simulation/runner.py` called `load_cifar10()` directly, so the `eurosat` and `custom` options already documented in `data/partition.py` and the README were dead code, never actually reachable regardless of the `data.dataset` config value. This is now fixed — `runner.py` calls the `load_dataset()` dispatcher, and the model's output layer size is inferred from the dataset (`_num_classes()`) rather than hardcoded to 10, so `eurosat` and the new `aider` option are both now actually usable, not just documented.

A misleading-metric finding. AIDER's `normal` class is 68% of the data. At the CIFAR-tuned hyperparameters (batch size 32), round 1 reported 68.1% accuracy and the run auto-declared convergence — but this is an artefact, not learning: a model predicting "normal" for everything scores ≈68% for free. We added a balanced-accuracy metric (`FLServer.evaluate_balanced()`, mean per-class recall) that a majority-class collapse cannot game, alongside the existing raw accuracy.

A privacy-budget finding. AIDER's clients hold far fewer samples than CIFAR's (∼257 vs. ∼2500 average per UAV), so the same batch size of 32 is a much higher DP-SGD sampling rate, and ε reached 9.05 of the 10.0 budget after a single round — reducing `local_batch_size` to 4 brought this qualitative problem under control (batch 32 exhausted the budget almost immediately; batch 4 did not). The specific ε figures quoted for this diagnostic run (2.17 at round 1, growing to 4.64 by round 10) predate the accounting fix in Section 8 and understate the true cost by roughly the same order of magnitude documented there; the qualitative finding — smaller batches reduce sampling-rate-driven budget exhaustion — is unaffected, since it compares two runs under the *same* (buggy, at the time) accounting, not an absolute number.

The remaining problem. Even with both fixes, the model does not learn. Table 5 shows raw accuracy oscillating wildly (8.6% to 67.5%) while balanced accuracy stays flat at 15–26% for all 10 rounds — essentially the 20% chance level for 5 classes, with no upward trend. The raw-accuracy swings are the model flipping in and out of a majority-class collapse, not genuine progress; balanced accuracy shows there is none.

Table 5: AIDER, 10-round diagnostic (batch size 4, otherwise identical DP-FL config to Section 8.1).

Round	1	3	5	7	9	10
Raw accuracy	8.6%	46.5%	66.4%	67.5%	67.3%	28.4%
Balanced accuracy	20.8%	25.9%	19.7%	20.0%	19.8%	20.8%

This was diagnosed as a data-scarcity problem compounding with the existing Dirichlet non-IID partitioning: with ∼257 images per UAV drawn from an already-imbalanced 5-class pool, a Dirichlet($\alpha = 0.5$) split across 20 clients can leave individual clients with few or zero examples of the minority disaster classes.

Stratified partitioning: implemented and tested. We verified the diagnosis directly: at $\alpha = 0.5$, **9 of 20 clients had zero examples of at least one class**. `dirichlet_partition()` (`data/partition.py`) now takes a `min_per_class` floor that reserves a minimum count of *every* class for *every* client before the Dirichlet-skewed remainder is distributed (default 0, i.e. the original behaviour, so CIFAR-10's reported results are unaffected). At `min_per_class=5`, the zero-class problem is eliminated entirely: **0 of 20 clients** are left without at least one example of every class.

Result: necessary, not sufficient. A 30-round run with the fix applied (batch size 4, $\sigma = 5.0$ — found the same way as Section 8.1's $\sigma = 6.6$, by binary search against the corrected accountant for a 30-round budget at AIDER's smaller per-client scale) still does not show genuine learning: balanced accuracy is 20.3% at round 1, 16.7% at round 10, 17.8% at round 20, and 21.2% at round 30 — flat around the 20% chance level for 5 classes, no trend, while raw accuracy climbs from 7.8% to 47.4% over the same rounds (the same majority-class drift balanced accuracy exists to catch, not real progress). A shorter check at the same batch size but the *original, uncorrected* noise level ($\sigma = 1.1$) did show balanced accuracy rising (23.9% → 26.0% over the first 10 rounds) before the correctly-accounted ε budget was exhausted at round 10 ($\varepsilon = 19.3$, almost 2× over budget) — i.e. the stratification fix genuinely helps, but not enough to survive the noise level a real $\varepsilon \leq 10$ guarantee requires at this cohort size.

This is the third independent line of evidence for the same conclusion as Sections 8.1 and 8.6: at this framework's UAV-scale cohort (11–20 active clients), a correctly-accounted DP-SGD privacy budget of $\varepsilon \leq 10$ requires noise that overwhelms the learning signal, regardless of backbone (Section 8.6), dataset (this section), or now data-engineering fix (stratified partitioning). The binding constraint is not any one of these; it is the cohort size itself, which is precisely why Section 9's future work leads with adaptive clipping and larger effective participation (Section 8.5) rather than further backbone or dataset changes.

9 Conclusion

This report has presented a unified framework for Task 3.2 of the COVER project, integrating FL, differential privacy, game-theoretic incentives, PoRS, a PoA blockchain, and a novel MARL agentic framework into a single simulation environment. Each UAV agent

independently learns an adaptive participation policy via DQN, with Q-networks periodically federated across the swarm. The MARL framework replaces static participation scheduling with emergent, state-aware behaviour that jointly optimises accuracy, privacy cost, energy consumption, and blockchain reward. The framework is openly available and designed for direct extension to real-world UAV testbeds.

Future work. (i) **Adaptive clipping and/or larger effective participation.** Three independent experiments (Sections 8.1, 8.6, 8.7) converge on the same root cause: at this framework's UAV-scale cohort (11–20 active clients), a correctly-accounted $\varepsilon \leq 10$ DP-SGD budget requires noise that overwhelms the learning signal, regardless of backbone, dataset, or partitioning fixes. This is the binding constraint, and the priority item for making any of the CIFAR-10, MobileNetV2, or AIDER results practically useful — via a dimensionality-aware technique such as per-layer or adaptive clipping [19], and/or increasing the number of clients contributing gradient signal per round, since DP-FedAvg-style noise is added once per round and divided by the participant count; (ii) Add DP protection to the Q-network federation step for formal policy privacy; (iii) replace PoRS simulation with SGX-based hardware attestation for deployment; (iv) extend convergence analysis to 3D trajectory optimisation jointly with power allocation (WP2 interface); (v) explore multi-task MARL where agents jointly optimise sensing, routing, and FL participation in a unified policy; (vi) class-weighted loss for AIDER, as a complement to the stratified partitioning already implemented (Section 8.7), once (i) makes the noise level survivable enough for either to be visible in the result.

Acknowledgements. This work was carried out at Northumbria University under the supervision of Professor Nauman Aslam, as part of the COVER project (Cooperative and Intelligent UAVs for Emergency Response Applications), funded by the European Union under the Marie Skłodowska-Curie Actions Staff Exchanges programme, Grant Agreement No. 101086228. Task 3.2 is led by Georg-August-Universität Göttingen (UGOE) with contributions from CentraleSupelec and UESTC.

References

[1] B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y Arcas, "Communication-efficient learning of deep networks from decentralized data," in *Proc. AISTATS*, 2017.

[2] M. Abadi *et al.*, "Deep learning with differential privacy," in *Proc. ACM CCS*, 2016.

[3] I. Mironov, "Rényi differential privacy," in *Proc. IEEE CSF*, 2017.

[4] Y.-X. Wang *et al.*, "Subsampled Rényi differential privacy and analytical moments accountant," in *Proc. AISTATS*, 2019.

[5] B. Balle, G. Berrada, and Y.-A. Le Borgne, "Hypothesis testing interpretations and renormalization of differential privacy," in *Proc. ICML*, 2020.

[6] Q. Yang *et al.*, "Incentive mechanism design for federated learning," *IEEE Trans. Wireless Commun.*, vol. 19, no. 8, 2019.

[7] Y. Zhan *et al.*, "A learning-based incentive mechanism for federated learning," *IEEE Internet Things J.*, vol. 7, no. 7, 2020.

[8] P. Blanchard, E. M. El Mhamdi, R. Guerraoui, and J. Stainer, "Machine learning with adversaries: Byzantine tolerant gradient descent," in *Proc. NeurIPS*, 2017.

[9] D. Yin, Y. Chen, K. Ramchandran, and P. Bartlett, "Byzantine-robust distributed learning: Towards optimal statistical rates," in *Proc. ICML*, 2018.

[10] C. Fung, C. J. M. Yoon, and I. Beschastnikh, "Mitigating sybils in federated learning poisoning," *arXiv:1808.04866*, 2018.

[11] A. Al-Hourani, S. Kandeepan, and A. Jamalipour, "Modeling air-to-ground path loss for low altitude platforms in urban environments," in *Proc. IEEE GLOBECOM*, 2014.

[12] V. Mnih *et al.*, "Human-level control through deep reinforcement learning," *Nature*, vol. 518, pp. 529–533, 2015.

[13] Z. Zheng, L. Zhao, J. Guo, W. Shi, and C. Lin, "Federated deep reinforcement learning for resource allocation in wireless networks," *IEEE Trans. Wireless Commun.*, vol. 22, no. 5, pp. 3382–3396, 2023.

[14] D. C. Nguyen *et al.*, "Federated learning for smart healthcare: A survey," *ACM Comput. Surv.*, vol. 55, no. 3, 2021.

[15] Y. Shi, K. Wei, L. Shen, Y. Liu, X. Wang, B. Yuan, and D. Tao, "Towards the flatter landscape and better generalization in federated learning under client-level differential privacy," *arXiv:2305.00873*, 2023.

[16] J. Liu, Y. Tian, F. Shang, Y. Liu, H. Liu, J. Zhou, and D. Ding, "DP-FedPGN: Finding global flat minima for differentially private federated learning via penalizing gradient norm," *arXiv:2510.27504*, 2025.

[17] O. I. Khalaf, S. R. Ashokkumar, S. Algburi, S. Anupallavi, D. Selvaraj, M. S. Sharif, and W. Elmedany, "Federated learning with hybrid differential privacy for secure and reliable cross-IoT platform knowledge sharing," *Security and Privacy*, vol. 7, no. 3, e374, 2024.

[18] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, "MobileNetV2: Inverted residuals and linear bottlenecks," in *Proc. IEEE CVPR*, 2018.

[19] G. Andrew, O. Thakkar, H. B. McMahan, and S. Ramaswamy, "Differentially private learning with adaptive clipping," in *Proc. NeurIPS*, 2021.